

De La Salle University
Software Technology Department

CSC611M

Machine Project 1: Parallel OCR Pipeline

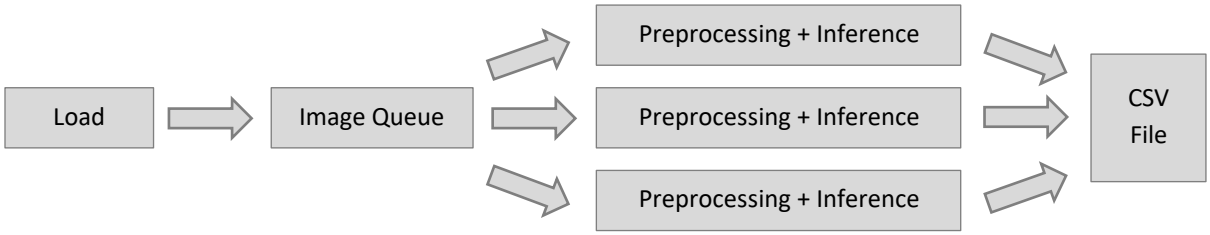
Description

An Optical Character Recognition (OCR) system is an application that extracts textual information from images. It has a wide range of uses in automation, document digitization, ID verification, license plate recognition, and other computer vision tasks. Modern OCR frameworks combine AI models with image processing techniques to achieve high accuracy even on noisy or low-quality inputs.

While these systems are effective, AI-based OCR can create performance bottlenecks as each image must be loaded, preprocessed, and passed to a model for inference. To improve efficiency, the sequential steps can be turned into a parallel pipeline that allows multiple images to be processed simultaneously.

Objective

Your task is to implement an OCR pipeline in Python that performs image preprocessing and inference in parallel, using workers that may be threads or processes. One worker loads the images and adds them to a shared queue. The remaining workers take images from the queue, preprocess them, and then run inference to extract the text. The results should be stored in a single CSV file (`ocr_results.csv`) containing the following details: ID (sequential integer), image filename, extracted text, and processing time (in milliseconds).



At the start of your program, a configuration text file is loaded (`config.txt`) which contains the directory of images to be processed and the number of workers (excluding the loader). The program should terminate once all images have been processed.

`config.txt`

```
image_dir=/your_image_directory
n_workers=2
```

You may use any synchronization construct from Python's standard library. Third-party libraries for concurrency or synchronization are not allowed. You are also free to use any OCR library of your choice. Tesseract is recommended because it is easy to use and lightweight, allowing efficient parallel execution. You're free to implement any preprocessing technique for the images. High accuracy is not the goal of this project, but at least half of the characters in the dataset should be recognized.

Report

Create a slide deck providing the following details:

- Project overview – A brief description of the OCR pipeline and its components.
- Parallelization – Explain how parallel computing was implemented in your application.
- Synchronization – Describe the synchronization constructs used, shared variables, and how workers were coordinated.
- Benchmarking – Experiment with varying numbers of workers and show the results. Indicate the performance gain per number of workers using the speedup formula. Identify the optimal configuration for your application.

$S = \frac{T_1}{T_n}$ T_1 total processing time with 1 worker

T_n total processing time with n workers

Deliverables

- Source code (zip file)
- Output CSV
- PPT file

Rubric

The project is to be graded using the following criteria:

CRITERIA	EXEMPLARY	SATISFACTORY	DEVELOPING	BEGINNING
Technical Documentation 10 %	**10**	**7**	**3**	**0**
	Clearly explains the pipeline architecture, the parallelization approach, and the synchronization design. Benchmarks varying numbers of workers, analyzes the results, and identifies the optimal configuration.	Mostly explains the architecture, parallelization, and synchronization, with minor gaps. Benchmarks are shown, but the analysis is limited.	The explanation of the architecture, parallelization, and/or synchronization is missing significant details. Benchmarks are incomplete, with little to no analysis.	No meaningful documentation
Parallel Techniques 40 %	**40**	**28**	**12**	**0**
	Multiple workers are created explicitly, and work is distributed through the shared queue as specified. All images are processed with complete output, and a performance gain is demonstrated.	Multiple workers are used, but the design departs slightly from the spec, work is distributed unevenly, and/or there is a minor bug that does not significantly affect the output.	Multiple workers are created, but the OCR pipeline is generally not performed in parallel, and/or there are major bugs in the implementation.	No meaningful implementation
Synchronization 50%	**50**	**35**	**15**	**0**
	Shared resources are protected and workers are properly coordinated. All workers terminate cleanly, with no race conditions or deadlocks.	Shared resources are protected, but coordination is inefficient and/or a minor synchronization issue exists.	Some shared resources are unprotected, and/or race conditions or deadlocks cause incorrect output or improper termination.	No meaningful implementation